Optimised Dense and Sparse Matrix Multiplication: Loop Order, Cache Blocking and CSR SpGEMM

Big Data - Individual Assignment 2 (Optimised approaches and sparse matrices)

Hugo Amador Hidalgo - Grado en Ciencia e Ingeniería de Datos, ULPGC - Academic year 2026-2027

Repository: https://github.com/hugoamadoor/BIG-DATA (folder Assignment02_AmadorHidalgo)

Abstract

This study asks two separate questions about sequential float64 matrix multiplication in C. Part A: how much do three optimisations - transposing B, reordering the loops to i-k-j, and cache blocking - speed up the Assignment 1 i-j-k baseline as n grows? Part B: at which tested densities and structures does a CSR sparse-sparse product (Gustavson's algorithm, own code) beat the best dense method? All methods were validated against NumPy/SciPy and are bit-identical to the baseline. On a 2-vCPU cloud VM (gcc 13, -O2, scalar code), the three optimisations were 11-17× faster than the baseline for $1024 \le n \le 2048$, mainly by removing the column-stride access to B. Blocking (tile 32) matched loop reordering up to n = 2048 and was 1.62× and 1.68× faster than it at n = 3072 and 4096. The baseline exceeded the 180 s limit at n = 3072; blocked was the only method above 3 GFLOP/s at n = 4096 (40 s). For uniformly random sparsity at n = 1024 and 2048, CSR SpGEMM was faster than dense blocked at every density up to 50 % and slower from 70 %, consistent with its d^2n^3 work and a cost per useful multiply-add about 3× higher than dense. Conversion from dense storage and fill-in reduced the advantage: at 0.1 % density the multiply alone was about 45 000× faster than dense, but only about 300× once the conversion was counted. At equal density, banded and power-law matrices ran up to 3.5× faster than uniform ones, so density alone did not determine the cost. With 16 nonzeros per row, CSR multiplied n = 131 072 matrices in 970 ms, while dense blocked hit the time limit at n = 8192.

1. Introduction and objective

Assignment 1 showed that the textbook i-j-k loop in C spends most of its time waiting for memory once the matrices no longer fit in cache: its inner loop walks B down a column, touching a new cache line on every iteration. This study keeps the same language, compiler flags, numeric type and inputs and asks how far that cost can be reduced without changing the $O(n^3)$ arithmetic, and when it is better not to store the zeros at all.

Research question A. For n × n dense float64 matrices, how do B-transposition, i-k-j loop reordering and cache blocking change runtime and memory relative to the Assignment 1 baseline, as n increases? **Research question B.** For matched logical inputs, at which densities and nonzero structures does CSR SpGEMM run faster than the best dense method, and how much of that advantage survives conversion costs and fill-in?

Hypotheses (stated before measuring). H1: any method that removes the stride-n access to B gives a large speed-up once B no longer fits in the caches. H2: blocking helps beyond loop reordering only when the data that i-k-j reuses between iterations of i (all of B) exceeds the effective cache. H3: SpGEMM wins below some density threshold; because its work is about d^2n^3 for uniform inputs but its cost per operation is higher, the threshold is well below 100 %, and conversion and fill-in lower its advantage.

2. Background

Locality and loop order. With row-major storage, i-j-k reads A with unit stride but B with stride n. Reordering to i-k-j turns the inner loop into C[i,:] += A[i,k]·B[k,:], so all three operands are read with unit stride [3], [4]. Transposing B first gives the same unit-stride access while keeping the i-j-k dot product. **Cache blocking** partitions the loops into tiles so that a tile of B is reused for many rows of A while it is still in cache; the best tile size depends on the cache hierarchy and is usually found empirically [5], [6]. Algorithmic alternatives such as Strassen's method [7] reduce the operation count but change the order of rounding and only pay off at large n, so they were not chosen here. Keeping the summation order fixed is what makes the bitwise comparison in Section 3.4 possible.

Sparse storage and SpGEMM. Compressed sparse row (CSR) stores for each row the column indices and values of its nonzeros, plus one row pointer per row [8], [9]. Gustavson's row-wise algorithm [10] computes C[i,:] as a combination of the rows B[k,:] selected by the nonzeros of A[i,:], gathering partial sums in a dense accumulator. Its work is the number of products $a_{ik}b_{kj}$ with both factors nonzero; for independent uniform patterns

with density d this is about d^2n^3, and the expected density of C is $1 - (1 - d^2)^n$, which is close to 1 once d^2n is a few units (fill-in). SciPy's sparse product, used here only as a reference, is based on SMMP [11], [12].

3. Methodology

3.1 Implementations

Method	Change relative to the baseline	Timed region
ijk (baseline)	Assignment 1 C kernel, copied unchanged (commit ca7622a); only restrict qualifiers added	loops
ijk_bt	B transposed into a scratch buffer, then i-j-k with unit stride in both operands	transpose + loops
ikj	loop reordering: C[i,:] += A[i,k] B[k,:]	zeroing C + loops
blocked	i-k-j inside square tiles (loop order ii, kk, jj, i, k, j); tile side 32 from experiment T	zeroing C + loops
csr_spgemm	CSR x CSR, row-wise Gustavson with a dense accumulator (two passes: symbolic, then numeric)	free old C + symbolic + allocation + numeric

Table 1. Methods. All are single-threaded C in src/dense.c and src/sparse.c; no library is called. Inputs and C (and the transpose buffer and accumulator) are allocated before the timer starts.

All dense methods use one contiguous row-major array per matrix, as in Assignment 1. CSR uses int64 row pointers, int32 column indices and float64 values (8(n+1) + 12·nnz bytes). Output column indices are left in production order (not sorted), which is valid CSR for this study but would need a sort for libraries that require ordered indices. Everything is compiled with gcc 13.3 -O2 -std=c11 -ffp-contract=off, the Assignment 1 flags. GCC enables limited auto-vectorisation at -O2 since version 12 [13], but the vectorisation report (build/vec_report_dense.txt) lists no vectorised loop in any kernel and the disassembly of ikj contains only scalar mulsd/addsd, so every result here is for scalar code. SIMD belongs to Assignment 3.

3.2 Inputs

Dense inputs are uniform in [0, 1), produced by NumPy's PCG64 generator with the Assignment 1 seeds, so they are byte-identical to the Assignment 1 files for the shared sizes. Sparse inputs are generated by scripts/gen_inputs.py (seed formula in the file); nonzero values are uniform in (0, 1], so no stored value is zero and no cancellation can occur. Three structures are used. **Uniform**: each entry is nonzero independently with probability d. **Banded**: every entry with $|i - j| \le w$, w chosen to match d. **Skewed**: row counts follow a 1/rank power law, ranks shuffled, rows capped at n. Sparsity is the percentage of zero entries and density the percentage of nonzeros; the achieved nonzero count of every file is in data/manifest.csv (112 files with SHA-256). For $n \le 2048$, and for n = 4096 and 8192 of experiment L, the same logical matrices are also written as dense files, so dense and sparse methods see identical inputs.

3.3 Experiments, environment and protocol

Id	Question	Configurations
T	tile side	blocked; n = 1024, 2048; tile 16, 32, 64, 128, 256. Rule declared in advance: lowest median at n = 2048
D	dense optimisations vs n	4 dense methods; n = 256, 512, 768, 1024, 1536, 2048, 3072, 4096
M	sparse vs dense, by density	uniform; n = 1024, 2048; d = 100, 85, 70, 50, 20, 10, 5, 2, 1, 0.5, 0.2, 0.1 %. SpGEMM (CSR input), SpGEMM (dense input: conversion timed separately), dense blocked; baseline ijk at n = 1024
S	structure at equal density	n = 2048, d = 5 % and 1 %; uniform, banded, skewed; dense blocked at 1 %
L	sizes beyond the dense budget	uniform, 16 nonzeros per row on average; n = 4096 to 131 072; dense blocked attempted until a limit is hit
X	cross-check	unchanged Assignment 1 binary vs baseline here; n = 512, 1024

Table 2. Experiments.

Hardware: Intel(R) Xeon(R) Processor @ 2.10GHz (KVM guest), 2 vCPUs, 1 thread per core, L1d 96 KiB (2 instances), L2 4 MiB (2 instances), L3 reported as 260 MiB (1 instance) (a value exposed by the hypervisor, not necessarily what this guest can use), 8220244 kB RAM; Linux-6.18.44-fc-v230-x86_64-with-glibc2.39; gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0. Every kernel is sequential; NumPy and SciPy are used only for inputs and validation and never run concurrently with a benchmark. One process runs one configuration. Timing uses clock_gettime(CLOCK_MONOTONIC) around the method only. One warm-up second was used for $n \le 1536$; for larger n no explicit warm-up was run, but one untimed calibration multiplication always precedes the samples. Ten measured samples for $n \le 512$, otherwise five (four for the baseline at n = 2048, where the 300 s sampling

budget stopped it after the minimum of three). Samples shorter than 0.1 s are batched and reported per multiplication. Budgets declared in advance: 180 s per multiplication, enforced inside the process with a timer signal; 15 min per configuration; 4 GiB of planned storage per process. A method that hits the time limit is not run at larger sizes. Statistic: median and inter-quartile range (IQR). Memory: peak resident set size (ru_maxrss) of the benchmark process, read with wait4() by a small wrapper, together with the analytic storage of the arrays. The runtime baseline of the programs, measured at n = 16, is 1.8 MiB.

3.4 Correctness

scripts/run_tests.py runs 205 checks (205 passed) before any benchmark. They cover a hand-computed 3×3 product, identity and zero matrices, an exact integer product, tile sizes that do not divide n (1, 7, 64, 256), CSR edge cases (empty rows, zero matrix, identity), and all three sparse structures. Every result must lie within $2\gamma_n|A||B|$ of NumPy, with $\gamma_n = nu/(1 - nu)$, $u = 2^{-53}$ [14]; the factor 2 covers the error of both our kernel and the BLAS reference. Because every method adds the products of C[i,j] in increasing k (zero products add exactly +0.0), the optimised dense methods and SpGEMM must also reproduce the baseline **bit for bit**; the number of nonzeros of C must equal SciPy's. During the benchmarks every process prints a checksum and three spot values, which the harness checks against NumPy or SciPy with the same tolerance; all completed runs passed.

4. Results

The full data set has 888 measured samples from 151 processes (results/raw_measurements.csv, runs.csv). One process from an aborted first launch of the harness is kept in the files under the label T_aborted and is excluded from the summaries. It had the same configuration as the first tuning run, and its last seconds overlapped with the correctness tests of the restarted launch, which is a further reason to exclude it.

4.1 Tile size (experiment T)

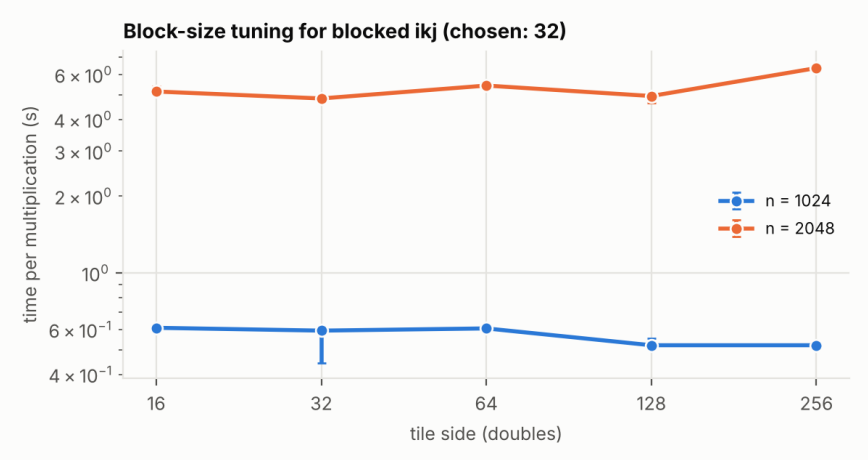


Figure 1. Blocked i-k-j against tile side. Median of 5 samples, IQR bars. The declared rule picked 32 (lowest median at n = 2048).

Tile side changed the blocked runtime by at most 32 % at n = 2048 (4.83 s to 6.35 s) and 17 % at n = 1024, with no monotonic trend: at n = 1024 tiles of 128 and 256 were fastest, at n = 2048 tile 256 was slowest. The chosen tile 32 and the runner-up 128 differed by 2 %, less than the spread between runs observed elsewhere, so the choice is weakly supported and was not tuned further. A 32×32 tile of each matrix is 24 KiB, which fits in the 48 KiB L1d of one core.

4.2 Dense methods (experiment D)

n	ijk (baseline)	ijk_bt	ikj	blocked (32)	blocked vs ikj	best GFLOP/s
256	17.1 ms ±3%	9.99 ms ±15% (1.7×)	7.19 ms ±4% (2.4×)	8.59 ms ±5% (2.0×)	0.84×	4.67
512	331 ms ±17%	96.6 ms ±8% (3.4×)	71.2 ms ±13% (4.6×)	69.3 ms ±13% (4.8×)	1.03×	3.87
768	709 ms ±14%	327 ms ±3% (2.2×)	280 ms ±18% (2.5×)	239 ms ±2% (3.0×)	1.17×	3.80
1024	9.16 s ±6%	821 ms ±6% (11.1×)	668 ms ±11% (13.7×)	605 ms ±13% (15.1×)	1.10×	3.55
1536	34.3 s ±2%	2.67 s ±2% (12.8×)	2.27 s ±7% (15.1×)	1.99 s ±8% (17.3×)	1.14×	3.65
2048	75.6 s ±2%	6.47 s ±4% (11.7×)	5.02 s ±1% (15.0×)	4.89 s ±1% (15.5×)	1.03×	3.51
3072	> 180 s (limit)	34.1 s ±1%	24.7 s ±25%	15.2 s ±2%	1.62×	3.81
4096	not attempted	77.9 s ±3%	67.3 s ±1%	40.1 s ±3%	1.68×	3.43

Table 3. Median time per multiplication ± IQR as % of the median; speed-up over the baseline in brackets. GFLOP/s = $2n^3$ / median time.

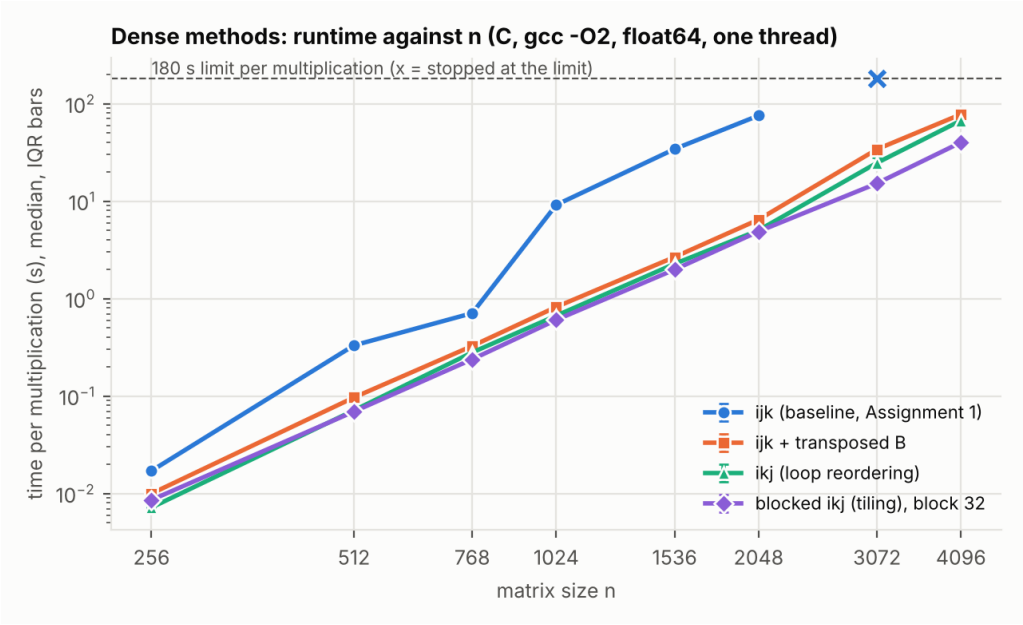


Figure 2. Dense runtime against n (log-log). The cross marks the baseline run stopped by the 180 s limit at n = 3072.

The column stride is the dominant cost. All three optimisations were 11× to 17× faster than the baseline for $1024 \le n \le 2048$, but only 1.7-2.4× at n = 256. The baseline itself jumped from 709 ms at n = 768 to 9.16 s at n = 1024 (13× the time for 2.37× the work), giving a fitted exponent of 4.2 for $n \ge 512$, against 3.04 for blocked. That jump at a power of two repeats the Assignment 1 observation that a row stride of 8192 bytes is especially costly for the i-j-k access to B; a cache-set-conflict explanation is plausible but was not measured with hardware counters here. H1 is supported.

Loop order versus transposition. ikj was faster than ijk_bt at every size (1.29× at n = 2048) although both read memory with unit stride and do the same arithmetic. ijk_bt also pays for the transposition, but that is $O(n^2)$ and small at these sizes. A more likely difference is that its inner loop is a reduction into one accumulator, so each addition waits for the previous one, while the ikj inner loop updates independent elements of C. This explanation is consistent with the data but was not isolated by a separate experiment.

Blocking only mattered at the largest sizes. Up to n = 2048 blocked and ikj were within 17 % of each other, and ikj was faster at n = 256. At n = 3072 and 4096 ikj fell to 2.35 and 2.04 GFLOP/s while blocked stayed at 3.81 and 3.43; blocked's cost per multiply-add stayed between 0.51 and 0.58 ns over the whole range. For every row i, ikj streams all of B, 32 MiB at n = 2048 and 72 MiB at n = 3072. The change of behaviour between these sizes is consistent with H2, an effective cache capacity between those two values. The hypervisor does not reveal how much of the reported 260 MiB L3 this guest can actually use, so this is an inference. ikj at n = 3072 also had the largest IQR of the study (25 %).

Memory. Peak RSS was the array storage plus the 1.8 MiB runtime baseline for ijk, ikj and blocked (for example 386 MiB at n = 4096, where $3 \cdot 8n^2$ = 384 MiB). ijk_bt needs one more n×n buffer (514 MiB). **Limits:** the largest size within the budget was n = 2048 for the baseline (n = 3072 hit the 180 s limit; 4096 not attempted) and n =

4096, the largest size attempted, for the three optimised methods.

4.3 Sparse versus dense on matched inputs (experiment M)

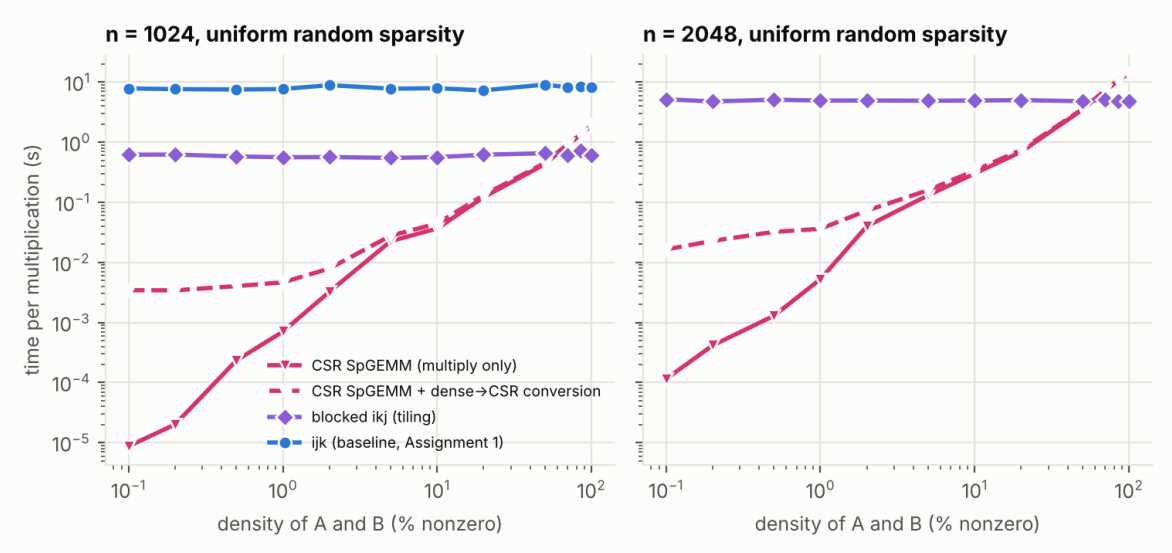


Figure 3. Time per multiplication against density (log-log), uniform random sparsity, same logical matrices for every method. Dense methods ignore the zeros, so their time is flat. The dashed line adds the dense-to-CSR conversion of A and B, measured separately.

d (%)	nnz(A)	nnz(C)	density C	SpGEMM	convert	dense blocked	speed-up	speed-up incl. conv.	ns / mult-add
100	4,194,300	4,194,300	100 %	14 s	69.9 ms	4.84 s	0.35×	0.34×	1.63
85	3,565,180	4,194,300	100 %	10.6 s	79.2 ms	4.79 s	0.45×	0.45×	1.71
70	2,934,610	4,194,300	100 %	7.21 s	81.3 ms	5.04 s	0.70×	0.69×	1.71
50	2,099,180	4,194,300	100 %	3.48 s	84.6 ms	4.81 s	1.38×	1.35×	1.62
20	838,908	4,194,300	100 %	693 ms	52.1 ms	5.01 s	7.23×	6.73×	2.02
10	419,049	4,194,300	100 %	301 ms	37.3 ms	4.93 s	16×	15×	3.51
5	209,721	4,168,820	99.4 %	131 ms	28.2 ms	4.9 s	37×	31×	6.11
2	84,328	2,352,080	56.1 %	40.1 ms	32.8 ms	4.94 s	123×	68×	11.61
1	41,786	777,844	18.5 %	5.26 ms	31 ms	4.93 s	937×	136×	6.11
0.5	20,860	207,387	4.94 %	1.29 ms	30.9 ms	5.09 s	3,934×	158×	6.09
0.2	8,421	33,683	0.803 %	415 µs	22.4 ms	4.77 s	11,480×	209×	12.28
0.1	4,050	8,370	0.2 %	115 µs	16.3 ms	5.17 s	44,866×	315×	13.76

Table 4. n = 2048, uniform. Medians of 5 samples (IQRs in results/summary_sparse.csv). Speed-up = dense blocked time / SpGEMM time; above 1 means sparse is faster. ns per mult-add = SpGEMM time divided by its actual scalar multiply-adds (Σ over nonzeros a_ik of nnz(B[k,:])).

Where sparse helps. At both sizes SpGEMM was faster than dense blocked at every density from 0.1 % to 50 % and slower at 70 % and above (n = 1024 showed the same pattern). Its time fell with density over more than five orders of magnitude, while the dense methods, which do not look at the values, stayed flat. That the break-even lies between 50 % and 70 % follows from the work model: SpGEMM performs about d^2n^3 multiply-adds at about 1.7 ns each in the dense-like regime ($d \geq 50$ %), against n^3 at about 0.54 ns for dense blocked, which predicts a break-even near $d = \sqrt{(0.54/1.71)} \approx 56$ %. This is a consistency check on the measured costs, not an independent prediction.

Where its costs reduce the advantage. (i) *Indexing.* The cost per useful multiply-add rose from about 1.6 ns at high density to 14 ns at 0.1 % (Table 4), because at low density per-row and per-nonzero bookkeeping (row pointers, the accumulator marker, writing output indices) is no longer amortised over many products, and each product needs an indirect access to the accumulator. (ii) *Conversion.* Converting A and B from dense to CSR took 16.3 ms to 84.6 ms at n = 2048, because it must scan all $2n^2$ entries whatever the density. At 0.1 % it was about 142× the multiply itself, and it cut the speed-up from 44,866× to 315×. Conversion is therefore worth paying only when the matrices already exist in sparse form or are reused. (iii) *Fill-in.* The product was much denser than its factors (Figure 4): at n = 2048, inputs with 1 % nonzeros gave 18.5 % nonzeros in C and inputs with 5 % gave

99.4 %, matching $1 - (1 - d^2)^n$ to within 2 %. The output then costs 12 bytes per entry in CSR instead of 8 in dense storage.

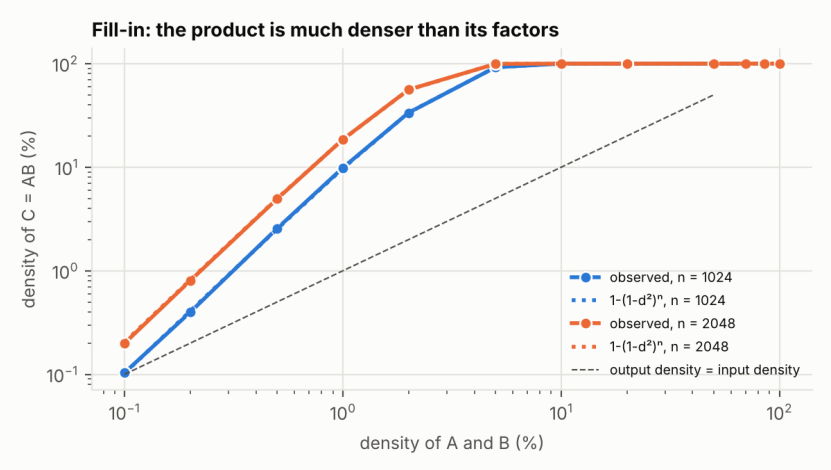


Figure 4. Density of the product against density of the factors, observed (solid) and the independent-pattern model $1 - (1 - d^2)^n$ (dotted, almost hidden by the solid line).

Memory. At n = 2048 SpGEMM's peak RSS was 2.0 MiB at 0.1 % and 11.7 MiB at 1 %, against 97.6 MiB for any dense method. The two met near 50 % (97.8 MiB), and at 100 % the sparse run needed 145.8 MiB, more than dense (Figure 5). Above about 2 % density, analytic CSR storage of A, B and C tracks the RSS closely; below it the fixed costs dominate (program baseline, row pointers, the 20n-byte accumulator). The memory advantage disappears together with the time advantage, at about the same density.

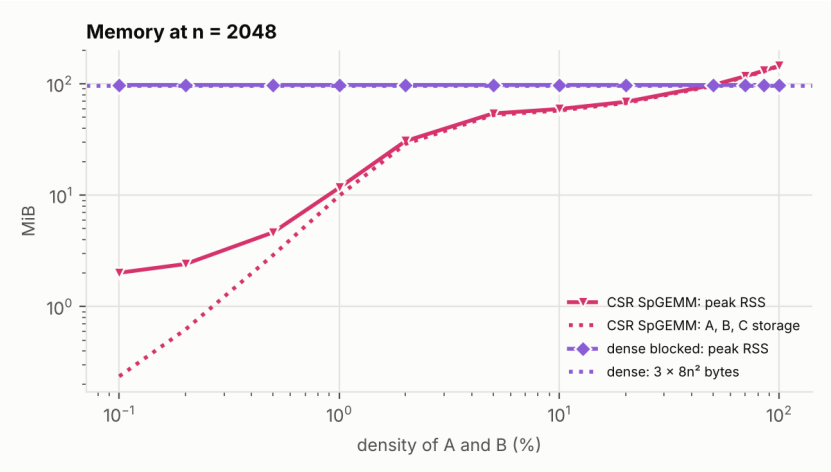


Figure 5. Peak RSS (solid) and analytic storage (dotted) at n = 2048.

4.4 Structure at equal density (experiment S)

structure	d (%)	nnz(A)	mult-adds	nnz(C)	density C	SpGEMM	ns / mult-add	peak RSS
uniform	5	209,721	21,438,600	4,168,820	99.4 %	124 ms	5.76	54.3 MiB
banded	5	208,292	21,272,000	409,334	9.76 %	35.4 ms	1.66	11.3 MiB
skewed	5	209,708	21,531,200	3,492,360	83.3 %	82.5 ms	3.83	46.5 MiB
uniform	1	41,786	860,492	777,844	18.5 %	5.59 ms	6.49	11.7 MiB
banded	1	42,898	899,318	83,548	1.99 %	2.41 ms	2.68	3.8 MiB
skewed	1	41,934	874,207	469,583	11.2 %	5.18 ms	5.92	8.2 MiB

Table 5. n = 2048, CSR input, median of 5 samples. Dense blocked on the 1 % matrices: uniform 5.07 s, banded 5.34 s, skewed 4.91 s.

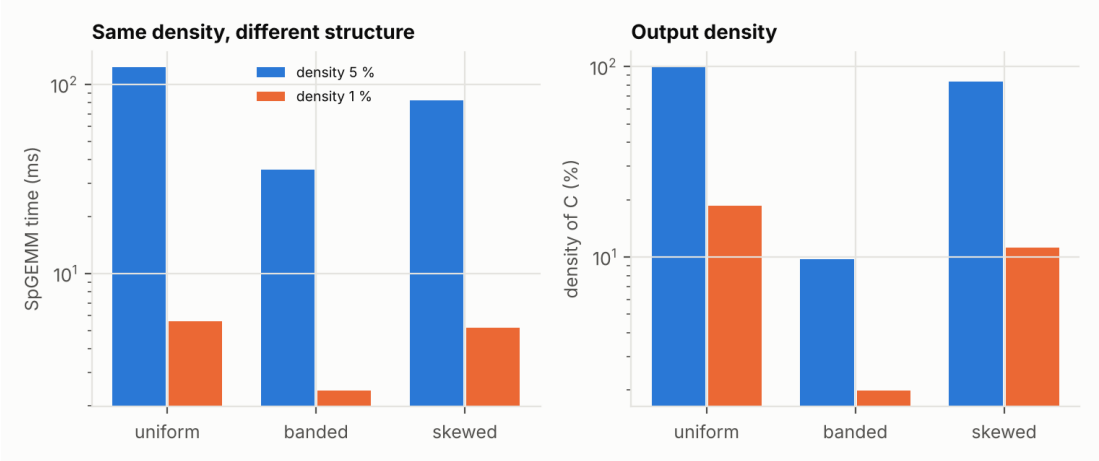


Figure 6. SpGEMM time and output density for three structures with the same input density.

At 5 % density the banded product took 35.4 ms and the skewed one 82.5 ms, against 124 ms for uniform. Two separate effects explain this. The amount of work and fill-in differ: the band of the product is only twice as wide as the band of the factors, so banded C had 9.8 % nonzeros where uniform C had 99.4 %. And the cost per multiply-add differs (1.66 ns banded vs 5.76 ns uniform). Neighbouring rows of a banded A select overlapping, neighbouring rows of B and a contiguous range of the accumulator, which is plausibly cache-friendly, although this was not measured directly. The skewed matrices concentrate nonzeros in a few long rows, which produced fewer multiply-adds and fewer output entries than uniform at the same density. The dense method was insensitive to structure (all three within 9 %). Density alone therefore did not determine sparse performance in these tests.

4.5 Sizes beyond the dense budget (experiment L)

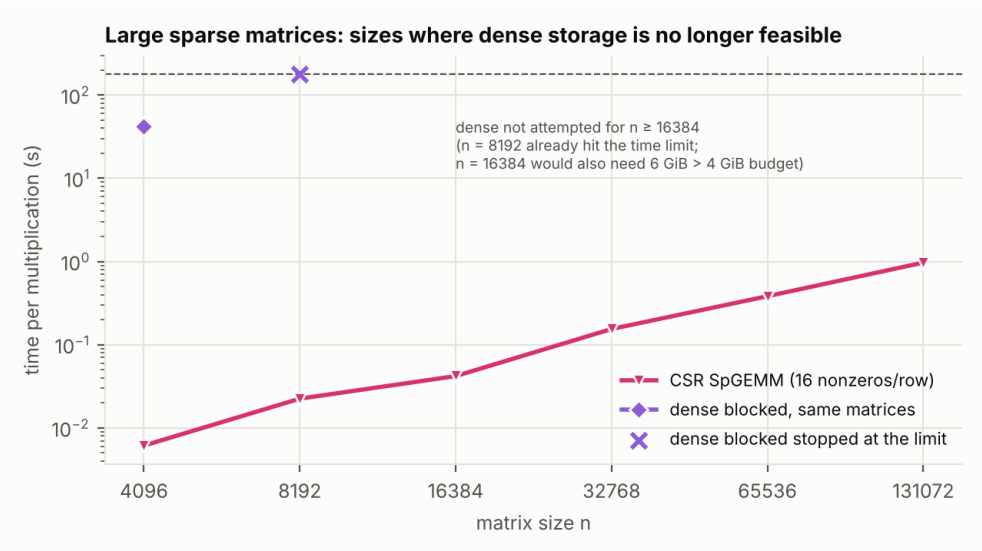


Figure 7. Uniform matrices with 16 nonzeros per row on average, so density falls as n grows.

n	density (%)	nnz(C)	SpGEMM	ns / mult-add	peak RSS	dense 3·8n²	dense blocked
4,096	0.391	1,020,740	6.13 ms	5.8	15 MiB	0.4 GiB	41.9 s
8,192	0.195	2,052,330	22.5 ms	10.8	29 MiB	1.5 GiB	> 180 s (limit)
16,384	0.0977	4,163,470	42 ms	10.0	56 MiB	6.0 GiB	not attempted
32,768	0.0488	8,338,800	155 ms	18.5	110 MiB	24.0 GiB	not attempted
65,536	0.0244	16,757,700	383 ms	22.8	220 MiB	96.0 GiB	not attempted
131,072	0.0122	33,492,700	970 ms	28.9	438 MiB	384.0 GiB	not attempted

Table 6. Experiment L. Dense blocked hit the 180 s limit at n = 8192 and was not attempted above. From n = 16 384 dense storage alone would also exceed the 4 GiB budget (6 GiB).

With a fixed number of nonzeros per row, the work grows only linearly in n (about 256n multiply-adds), yet the measured time grew as $n^{1.45}$. The cost per multiply-add rose from 5.8 to 28.9 ns, which is consistent with

irregular accesses to B rows and to an accumulator that grows with n (2.5 MiB at n = 131 072) and stops fitting in the faster caches. The largest size tested, n = 131 072, completed in 970 ms with 438 MiB. That was the largest size declared, not a limit reached: no sparse configuration hit a budget. Dense storage at that size would need 384 GiB.

4.6 Cross-check with the Assignment 1 binary (experiment X)

The unchanged Assignment 1 binary took 7.65 s at n = 1024 and the baseline here took 7.58 s, so adding restrict did not change the baseline at that size. At n = 512 the two differed by 21 %. The Assignment 1 report measured 4.55 s at n = 1024 for the same binary and inputs on a different VM instance. Across the 14 separate processes that ran the baseline at n = 1024 in this study, the median ranged from 7.29 s to 9.16 s. That spread between processes (26 %) is larger than the within-run IQRs, so the baseline speed-ups should be read with about ±13 % uncertainty, and absolute times should not be compared with Assignment 1.

5. Discussion

Answer to question A. On this machine the decisive optimisation was removing the stride-n access to B. Any of the three methods that do so gave an order-of-magnitude speed-up once $n \geq 1024$. Beyond that, loop reordering beat transposition by about 29 %, and blocking added a further 68 % only when B exceeded the data that apparently stayed cached ($n \geq 3072$). All gains here came from memory behaviour: the arithmetic and its order are identical in all four methods (the outputs are bit-identical) and the code is scalar. Even the best method reached only about 3.5-3.9 GFLOP/s, a small fraction of what SIMD, register blocking and multiple threads could deliver [6]; those belong to Assignment 3.

Answer to question B. For uniform random patterns, CSR SpGEMM was faster than the tuned dense method at all densities up to 50 % and slower from 70 % at both tested sizes. Its advantage grew steeply as density fell because the work falls with d^2. That advantage is reduced, and at high density reversed, by (i) a cost per operation between about 3× and 25× that of dense blocked, (ii) the conversion from dense storage, which scans all n^2 entries and dominated below about 1 %, and (iii) fill-in, which makes C nearly dense at moderate input densities and removes the memory advantage. Structure mattered as much as density: at the same 5 % density the banded inputs ran about 3.5× faster and produced a ten times sparser output than the uniform ones. Optimisation effects (Part A) and storage effects (Part B) are separate here: the dense comparison used the best tested dense method, and the sparse kernel received no tuning.

Bottlenecks observed. Baseline: column-stride memory access, worst at power-of-two n. ikj: streaming the whole of B for every row once B no longer fits in cache. ijk_bt: probably the serial dependence of the dot product. SpGEMM: indirect access to the accumulator and per-row overheads at low density, conversion cost when inputs start dense, and growing cache misses as n grows at fixed nonzeros per row.

Limitations. (1) One 2-vCPU cloud VM, whose cache hierarchy is only partly visible; the run-to-run spread of the baseline was larger than the within-run IQR, and the results do not transfer directly to other hardware such as the author's laptop. (2) No hardware performance counters were available (perf is not installed), so the cache explanations are inferences from timing and data sizes, labelled as such. (3) The sparse inputs are synthetic. A real matrix from the SuiteSparse collection [15] would add realism, but the collection could not be downloaded from the benchmark machine; the CSR format used here would accept it after conversion. (4) Only one sparse algorithm and one format were studied, with unsorted output columns, and SciPy was used only for checking, not as a timed competitor. (5) The tile size was tuned once on two sizes, and differences between tiles were close to the noise. (6) Scalar code only, -O2, no -march.

6. Conclusions

Within the tested scope (C, gcc -O2, float64, one thread, one cloud VM, synthetic inputs), the study establishes four points. Making the inner loop read B with unit stride turned a baseline that ran at about 0.2 GFLOP/s for $n \geq 1024$ into methods running at 2.6-3.6 GFLOP/s, and it moved the largest size within the 180 s limit from 2048 to at least 4096. Cache blocking was only needed for the largest sizes tested, where it kept the rate near 3.4-3.8 GFLOP/s while plain loop reordering dropped to about 2. CSR SpGEMM beat the best dense method for uniform inputs up to 50 % density, but conversion, fill-in and structure changed the size of that advantage by orders of magnitude, so the share of zeros alone is not a sufficient predictor. CSR alone made n = 131 072 feasible in about one second and under 0.5 GiB. Still open: the exact break-even density between 50 % and 70 %; hardware-counter evidence for the cache explanations; real-world matrices; and how these results change with vectorisation and threads, which Assignment 3 will address.

7. Use of generative AI

In line with the course policy, the AI assistant Claude (Anthropic, claude.ai) was used to plan the experiments, draft the C implementations, the harness and analysis scripts, the README and the first draft of this report. All numerical results come from measurements stored in results/ and produced by the committed code; the AI output is not used as evidence. The author remains responsible for understanding the code, validating the measurements, verifying the references and defending the conclusions.

References

[1] M. D. Afonso Suárez, J. J. Hernández Cabrera and J. Évora Gómez, “Big Data,” course materials, Universidad de Las Palmas de Gran Canaria, 2026/27. [Online]. Available: https://mltechdrawer.github.io/BIGDATA/

[2] “Individual Assignments: Matrix Multiplication,” Big Data, Grado en Ciencia e Ingeniería de Datos, ULPGC, revised proposal 2026-2027.

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Nov. 2007.

[5] M. S. Lam, E. E. Rothberg and M. E. Wolf, “The cache performance and optimizations of blocked algorithms,” in *Proc. ASPLOS IV*, 1991, pp. 63-74.

[6] K. Goto and R. A. van de Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Trans. Math. Softw.*, vol. 34, no. 3, art. 12, 2008.

[7] V. Strassen, “Gaussian elimination is not optimal,” *Numerische Mathematik*, vol. 13, pp. 354-356, 1969.

[8] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[9] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2006.

[10] F. G. Gustavson, “Two fast algorithms for sparse matrices: Multiplication and permuted transposition,” *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250-269, 1978.

[11] R. E. Bank and C. C. Douglas, “Sparse matrix multiplication package (SMMP),” *Advances in Computational Mathematics*, vol. 1, pp. 127-137, 1993.

[12] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261-272, 2020.

[13] “GCC 12 Release Series: Changes, New Features, and Fixes” (vectorisation enabled at -O2), Free Software Foundation. [Online]. Available: https://gcc.gnu.org/gcc-12/changes.html

[14] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[15] T. A. Davis and Y. Hu, “The University of Florida sparse matrix collection,” *ACM Trans. Math. Softw.*, vol. 38, no. 1, art. 1, 2011.

[16] H. Amador Hidalgo, “Basic matrix multiplication in Python, Java and C,” Big Data, Individual Assignment 1, ULPGC, 2026.

Reproducibility materials

Repository: https://github.com/hugoamadoor/BIG-DATA, folder Assignment02_AmadorHidalgo. It contains src/ (dense.c, sparse.c, harness.h), build.sh, a README with exact commands, requirements.txt, scripts/ (input generator, tests, benchmark harness, analysis, this report), data/manifest.csv (seeds, nonzero counts and SHA-256 of every input), and results/ (raw samples, one row per process, summaries, key numbers, environment, correctness log and figures). The inputs (about 3 GB) are regenerated with scripts/gen_inputs.py. The baseline kernel is reused from Assignment01_AmadorHidalgo/src/c/matmul.c in the same repository [16].